Link do repozytorium: https://github.com/pattosz/TM-95123

5.1 Capabilities Discovery

Po co automatyzujemy "Discovery" (Zadanie 5.1)?

1. Eliminacja "Zgaduj-Zgadula": Tester manualny często szuka nazwy paczki i aktywności w ustawieniach telefonu lub internecie. Inżynier wyciąga je bezpośrednio z kodu źródłowego (Manifestu). To jedyne pewne źródło.

2. Obsługa wielu wersji (Versioning): Aplikacje mobilne zmieniają się co tydzień. Ręczne wpisywanie appActivity przy każdej zmianie to pewny błąd. Skrypt 5.1 sprawia, że testy są zawsze aktualne, bo czytają dane z aktualnie testowanego pliku APK.

3. CI/CD Ready: W profesjonalnym procesie (Continuous Integration) nikt nie wpisuje parametrów ręcznie. Skrypt musi sam "rozpoznać" aplikację, którą ma przetestować.

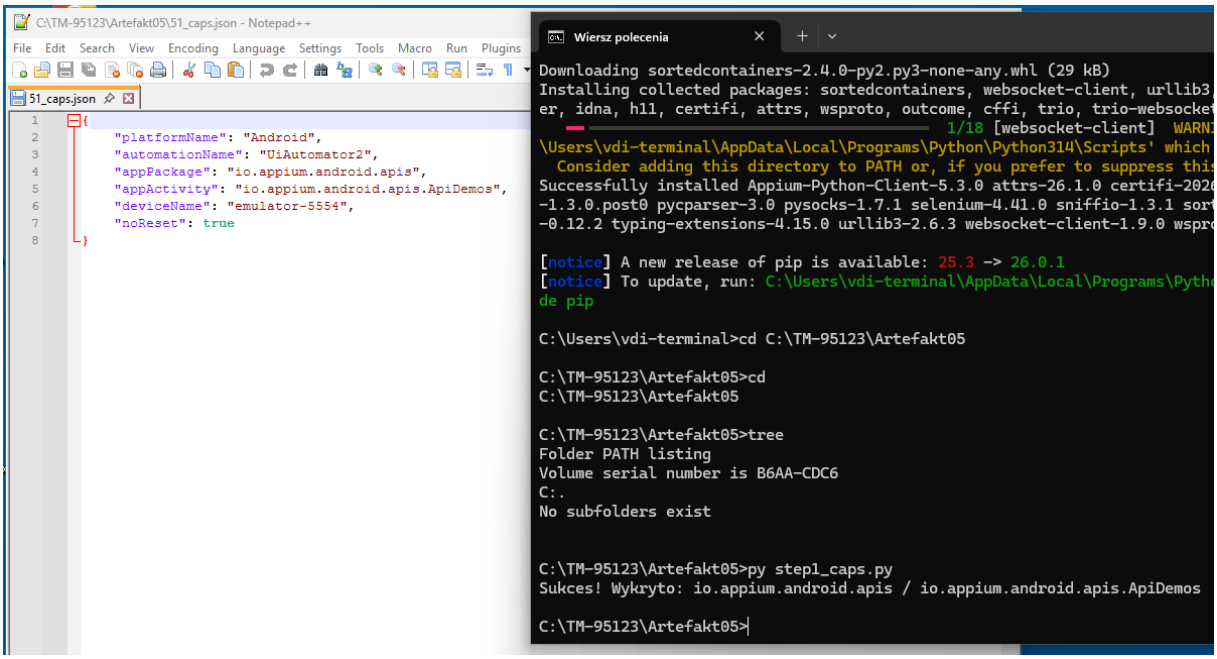
4. Zrozumienie Architektury Androida: Uczymy się, że każda aplikacja ma "drzwi wejściowe" (Main Activity). Bez ich znajomości Appium nie wie, od którego ekranu zacząć pracę.

Zadanie: Napisz skrypyt step1_caps.py który sam idzie do Artefaktu 02, czyta manifest i szuka aktywności, która ma filtr MAIN i LAUNCHER. Wynik niech zapisze w pliku: 51_caps.json

INSTRUKCJA DLA STUDENTA (ZADANIE 5.1)

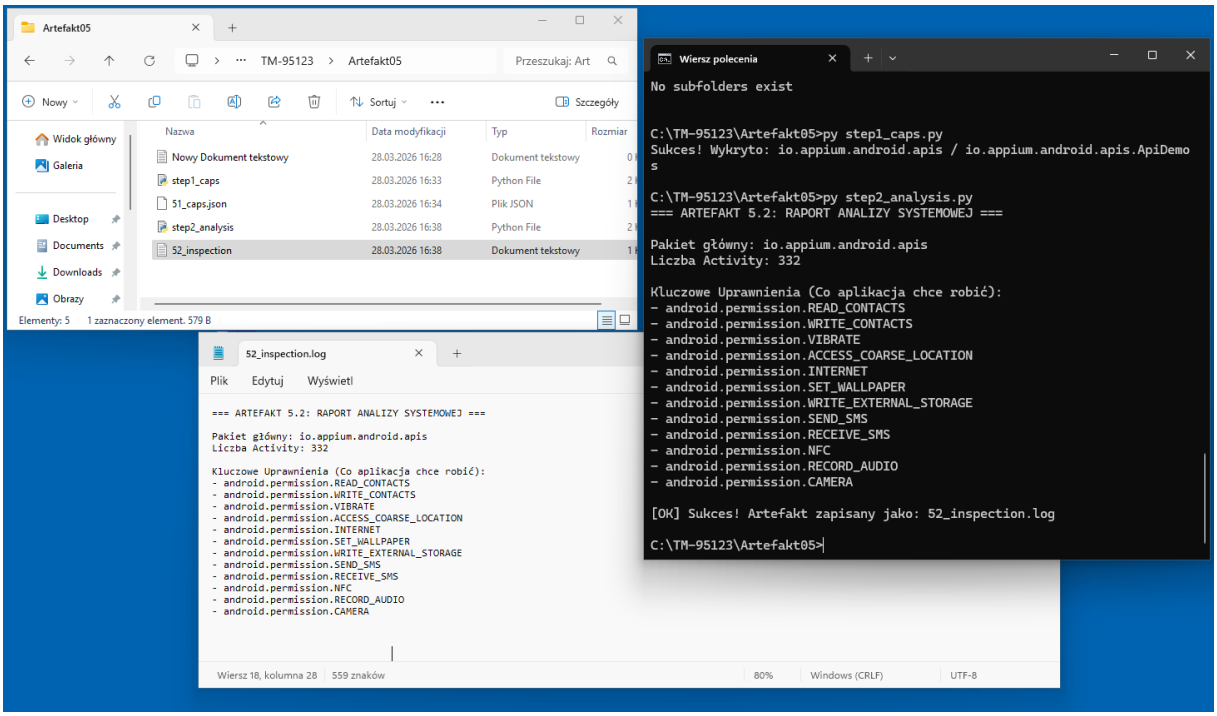
1. Upewnij się, że Twoje pliki z Bloku 2 są poprawnie zdekodowane i znajdują się w folderze ../Artefakt02/decompiled_apk/.

2. Uruchom skrypt: Mac: python3 step1_caps.py. Windows: python step1_caps.py. 3. Kiedy zobaczysz ramkę z napisem Sukces! Wykryto: ……. ….. …. , wykonaj screenshot całego okna terminala.



5.2. Programowy Inspektor Manifestu (Co robimy) Polecenie: Napisz program w języku Python, który dokona automatycznej inspekcji rozpakowanego manifestu. Program musi wyciągnąć kluczowe dane potrzebne do konfiguracji Appium i zapisać je w logu..

Cecha	Analiza Statyczna (Zadanie 5.2)	Analiza Dynamiczna (Appium)
Koszt	Prawie zerowy (brak infrastruktury)	Wysoki (wymaga emulatora/kabla)
Szybkość	Milisekundy	Minuty (bootowanie systemu)
Co badamy?	Deklaracje, uprawnienia, ukryte ekrany	Realne zachowanie, UX, wydajność
Zastosowanie	Wczesne wykrywanie błędów konfiguracji	Testy regresji i akceptacyjne

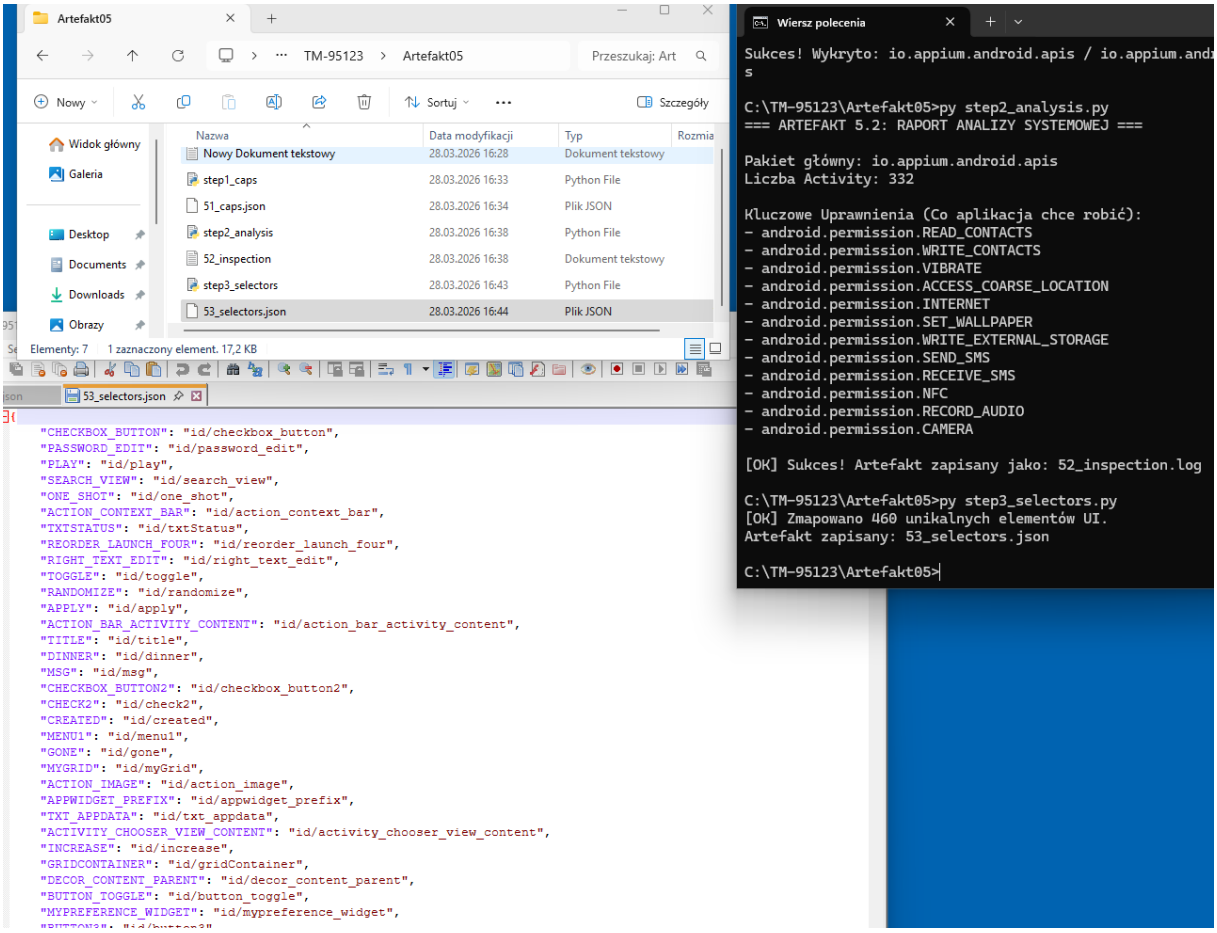


WSTĘP 5.3 – Mapa Interfejsu (UI Mapping)

W poprzednich krokach dowiedzieliśmy się, jak aplikacja się nazywa i jakie ma uprawnienia (5.2). Teraz musimy wiedzieć, w co "klikać". Ale zamiast błądzić po aplikacji, stworzymy Słownik Selektorów. Po co to robimy? Wyobraź sobie, że programista zmienia ID przycisku z btn_login na button_submit. Jeśli masz to ID wpisane w 100 testach – wszystkie padną. Jeśli masz je w jednym Słowniku Selektorów – zmieniasz jedną linię i wszystko działa. To się nazywa Maintainability (utrzymywalność) kodu.

Cecha	Automated ID Mining (Blok 4)	UI Mapping & Page Objects (Blok 5)
Cel	Znaleźć jakikolwiek działający identyfikator.	Stworzyć mapę biznesową dla frameworka.
Narzędzia	Szybki skan plików, grep, proste skrypty.	Klasy Pythona, Słowniki (JSON), Mapy.

Kontekst	Czy to ID w ogóle istnieje?	Jak to ID nazywa się w języku biznesowym?
Przykład	"Znalazłem id/txt_01"	"LOGIN_FIELD to id/txt_01"



WSTĘP 5.4 – Budowa Session Managera Do tej pory zbierałeś dane. Teraz czas je ożywić. W tym zadaniu stworzysz skrypt, który zachowuje się jak prawdziwy framework testowy:

1. Wczytuje konfigurację z 51_caps.json.

2. Pobiera selektory z 53_selectors.json.

3. Inicjalizuje połączenie z serwerem Appium (Docker).

Dlaczego to jest ważne? Jeśli zmieni się urządzenie, zmieniasz tylko plik JSON. Jeśli zmieni się selektor, zmieniasz tylko mapę. Kod testu pozostaje nienaruszony. To jest esencja Automatyzacji Klasy Enterprise.

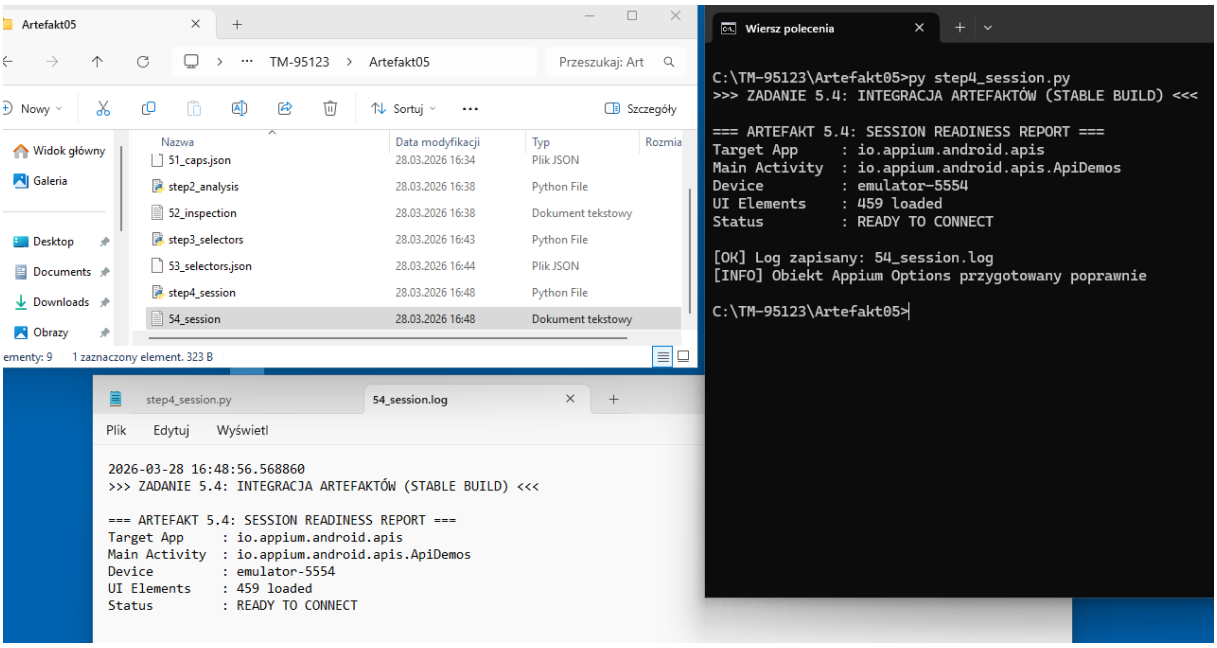
ZADANIE 5.4: Budowa Session Managera

Polecenie: Napisz skrypt Python, który dokona integracji wszystkich przygotowanych wcześniej artefaktów. Skrypt musi wczytać dane z plików JSON i na ich podstawie przygotować obiekt Options dla Appium.

Wymagania:

1. Skrypt musi połączyć dane z 51_caps.json oraz mapę z 53_selectors.json.

2. Wynik pomyślnego przygotowania sesji (możesz zrobić fizyczne połączenie na porcia appium lub konieczności fizycznego połączenia, korzystając z porzednich rezulatatów) ma zostać zapisany w 54_session.log.



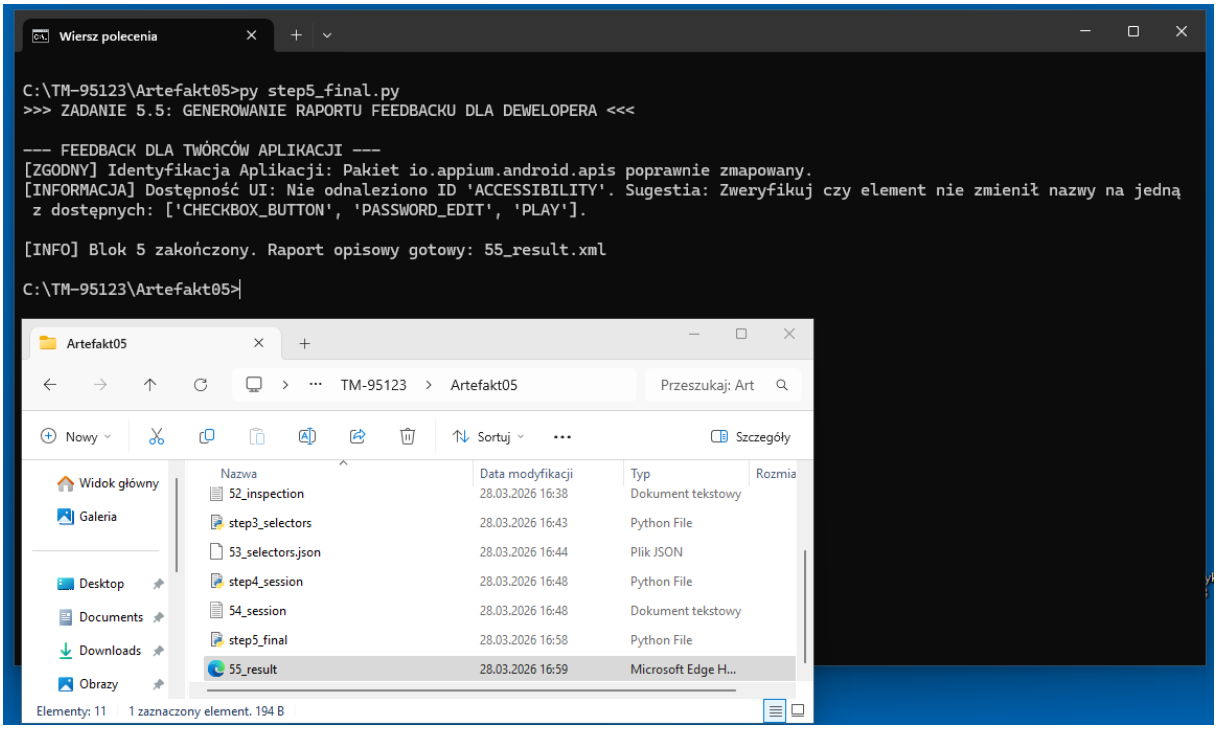
5.5 Asercja i Raport końcowy
Dotarliśmy do końca cyklu przygotowania testu. W świecie QA (Quality Assurance) samo "uruchomienie" czegoś to za mało. Musimy wygenerować dowód zaliczenia testu, który może przeczytać maszyna (np. serwer Jenkins).

Dlaczego generujemy plik XML? Standard JUnit XML to uniwersalny język raportowania. Dzięki niemu menedżerowie i programiści widzą zielone lub czerwone kafelki w systemach CI/CD bez zaglądania do Twojego kodu.

ZADANIE 5.5: Test Runner i Walidacja Krzyżowa

Polecenie: Napisz skrypt step5_final.py. Twoim zadaniem jest przeprowadzenie "testu spójności" (Consistency Test). Musisz sprawdzić, czy dane, które zdefiniowałeś jako tester (5.1), pokrywają się z rzeczywistością, którą wyciągnąłeś z plików aplikacji (5.2 i 5.3).

Wymagania:

1. Skrypt musi zweryfikować, czy appPackage z konfiguracji zgadza się z identyfikatorem z aplikacji.
2. Skrypt musi sprawdzić, czy kluczowy element interfejsu (np. ACCESSIBILITY) znajduje się w Twojej mapie selektorów.

3. Wynik musi zostać zapisany w pliku 55_result.xml.